

Infra-Deployment Pipeline

Objective:

The objective of this project is to design and implement a fully automated Infrastructure Deployment Pipeline that streamlines the provisioning, configuration, and management of cloud resources. By leveraging Infrastructure as Code (IaC) and CI/CD practices, the pipeline ensures consistency, repeatability, scalability, and reliability of infrastructure deployments across multiple environments. This reduces manual errors, accelerates release cycles, and establishes a standardized process for managing infrastructure in alignment with DevOps best practices.

Steps:

i. Create Folders in Visual Studio:

- To implement the Jenkins automation, first the Terraform code need to be created which has AWS Configurations.
- So, Open Visual Studio Code -> Open Folder -> Choose the folder named with “DevOps Project”.
- Under that create a folder named with “modules”
- So first to run the terraform code, extensions need to be installed -> Terraform (Anton Kulikov).
- After that under “modules” folder create two folders named with “vpc” & “ec2”.
- In those both folders create main.tf and var.tf files to run the code in the modules.
- At first the folders with files will be like this,

```
modules/  
|  
|----vpc/  
|   |----main.tf  
|   |----var.tf  
|  
|----ec2/  
|   |----main.tf  
|   |----var.tf
```

- After that, go to registry.terraform.io -> then choose the cloud provider -> choose the AWS provider -> then copy the codes for EC2 and VPC to create services in the AWS console automatically.
- So, the codes for EC2 and VPC are:

→ **Codes:**

▪ **VPC:**

--- **main.tf:**

```
resource "aws_vpc" "myvpc" {
  cidr_block      = var.vpc_cidr
  instance_tenancy = "default"
```

```
  tags = {
    Name = "${var.env}-VPC"
  }
}
```

```
resource "aws_subnet" "mysubnet" {
  vpc_id      = aws_vpc.myvpc.id
  cidr_block  = var.subnet_cidr
  availability_zone = var.az
```

```
  tags = {
    Name = "${var.env}-Subnet"
  }
}
```

```
output "subnet_id" {
  value = aws_subnet.mysubnet.id
  description = "The ID of the created subnet"
}
```

--- **var.tf:**

```
variable "vpc_cidr" {
  description = "CIDR block for the VPC"
  type        = string
}
```

```
variable "env" {
  description = "env we're going to create"
  type        = string
}
```

```

variable "subnet_cidr" {
  description = "CIDR block for the subnet"
  type       = string
}

variable "az" {
  description = "Availability zone for the subnet"
  type       = string
}

```

- **EC2:**

```

---main.tf:
resource "aws_instance" "web" {
  ami          = var.ami_id
  instance_type = var.instance_type
  subnet_id    = var.subnet_id
  count        = var.ec2_count

  tags = {
    Name = "${var.env}-EC2-${count.index}"
    Env  = var.env
  }
}

--- var.tf:
variable "ami_id" {
  type       = string
  description = "The AMI ID for EC2 instance"
}

variable "instance_type" {
  type       = string
  description = "Type of EC2 instance"
}

variable "subnet_id" {
  type       = string
  description = "Subnet ID of EC2"
}

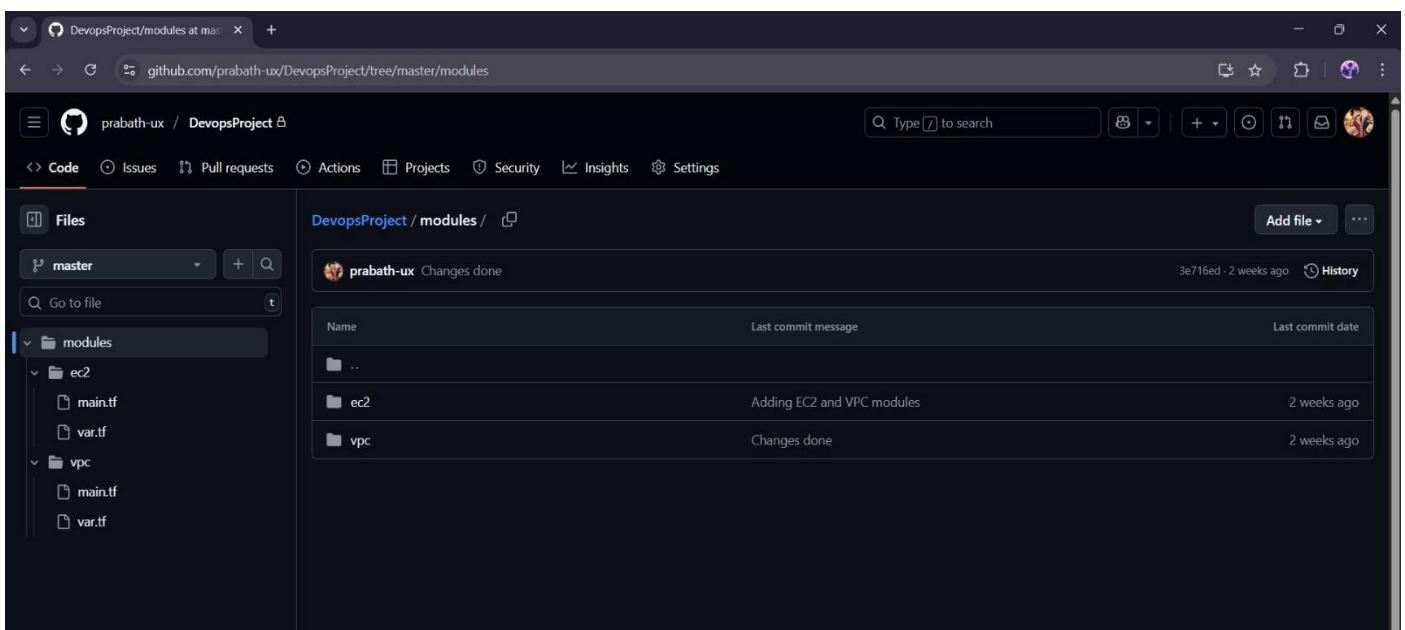
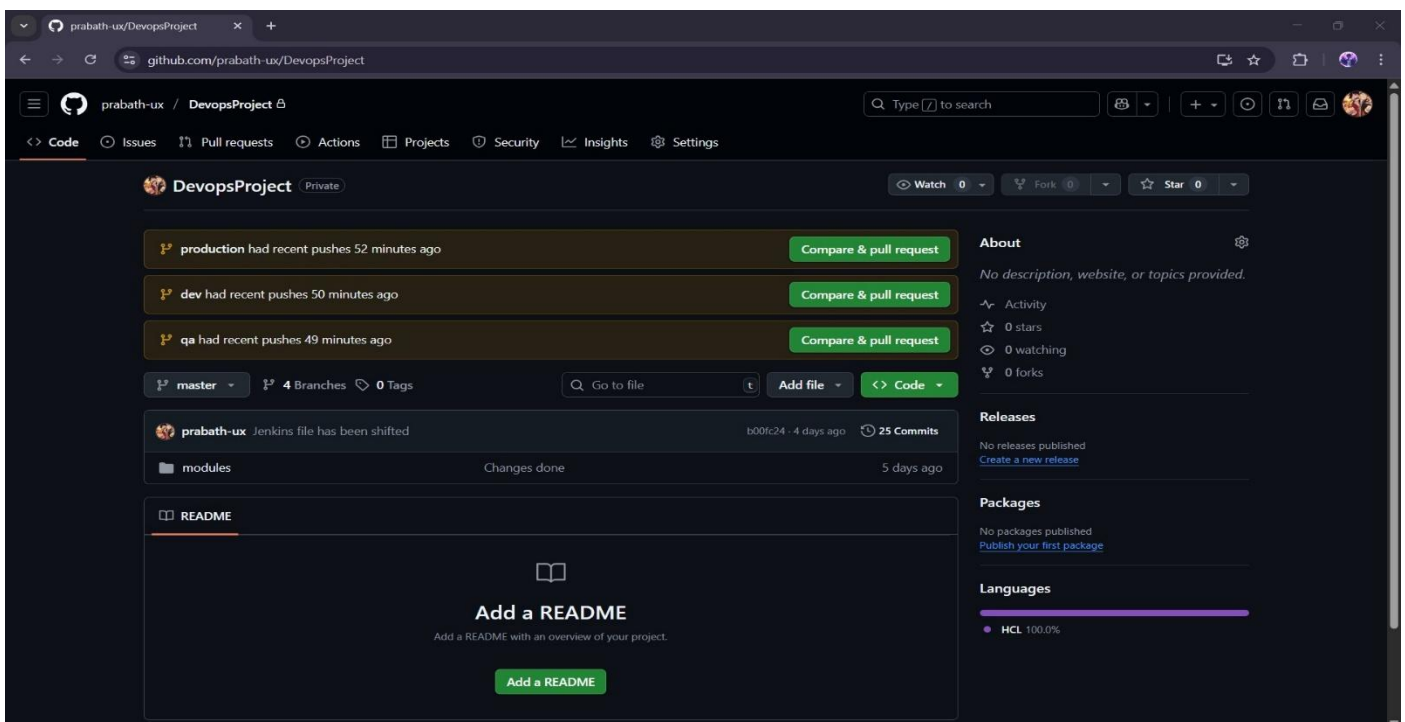
variable "ec2_count" {
  type       = number
  description = "The number of EC2 instances"
}

```

```
}
```

```
variable "env" {  
  type      = string  
  description = "Environment name (dev, qa, prod)"  
}
```

- So, after the creation of code give the following commands to push into the master branch.
- The following commands are:
 - git add .
 - git commit -m "The codes for master branch"
 - git push origin master
- And this is how it looks like in the GitHub page for the master branch.



- Now, create another directory in the Visual Studio Code to access the folders named with dev, qa, and production.
- This is how the multi-branch pipeline is going to be implemented to build and automate.
- So, the directory with the folders is created like this,

```
environments/
|
|----dev/
|   |--backend.tf
|   |--main.tf
|
|----qa/
|   |--backend.tf
|   |--main.tf
|
|----production/
|   |--backend.tf
|   |--main.tf
|----Jenkinsfile
```

- Now, the codes need to be created and modified in each branch.
- But first we need to create a repository in GitHub to save these folders and files in that.
- So, for that, go to github.com -> Create a Repository -> Name as “DevOps Project” -> Set it as Private -> Save it.
- Then copy the git remote origin <<https://github.com.username.project-name> > and paste it in the vs code after opening and installing the git software in the bash terminal.
- It will ask for password give the PAT token that is created through GitHub.
- Then it will start off with the master branch. From here on we can shift from one branch to the other.
- So, the codes need to be imported to run the commands, to switch branch from one to another use ‘git checkout <branch-name>’.

→ Codes:

- **dev:**
 - git checkout dev
 - --backend.tf:
 - terraform {
 - required_providers {

```

aws = {
  source = "hashicorp/aws"
  version = "5.100.0"
}
}
backend "s3" {
  bucket = "devopsproject16082025"
  key    = "environments/dev/terraform.tfstate"
  region = "ap-south-1"
}
}

```

```

provider "aws" {
  region = "ap-south-1"
}

```

```

--main.tf:
module "vpc" {
  source      = "../modules/vpc"
  vpc_cidr    = "10.0.0.0/16"
  env         = "dev"
  subnet_cidr = "10.0.1.0/24"
  az          = "ap-south-1a"
}

```

```

module "ec2" {
  source      = "../modules/ec2"
  instance_type = "t2.medium"
  ami_id      = "ami-0f918f7e67a3323f0"
  env         = "dev"
  ec2_count    = 3
  subnet_id    = module.vpc.subnet_id
}

```

```

--Jenkinsfile:
pipeline {
  agent any

  environment {
    ENV = "${env.BRANCH_NAME}"
    TF_WORKDIR = "environments/${env.BRANCH_NAME}"
  }
}

```

```

stages {

```

```

stage('Checkout') {
    steps {
        git branch: "${env.BRANCH_NAME}",
            credentialsId: 'git-creds',
            url: 'https://github.com/prabath-ux/DevopsProject.git'
    }
}

stage('Terraform Init') {
    steps {
        dir("${TF_WORKDIR}") {
            sh 'terraform init'
        }
    }
}

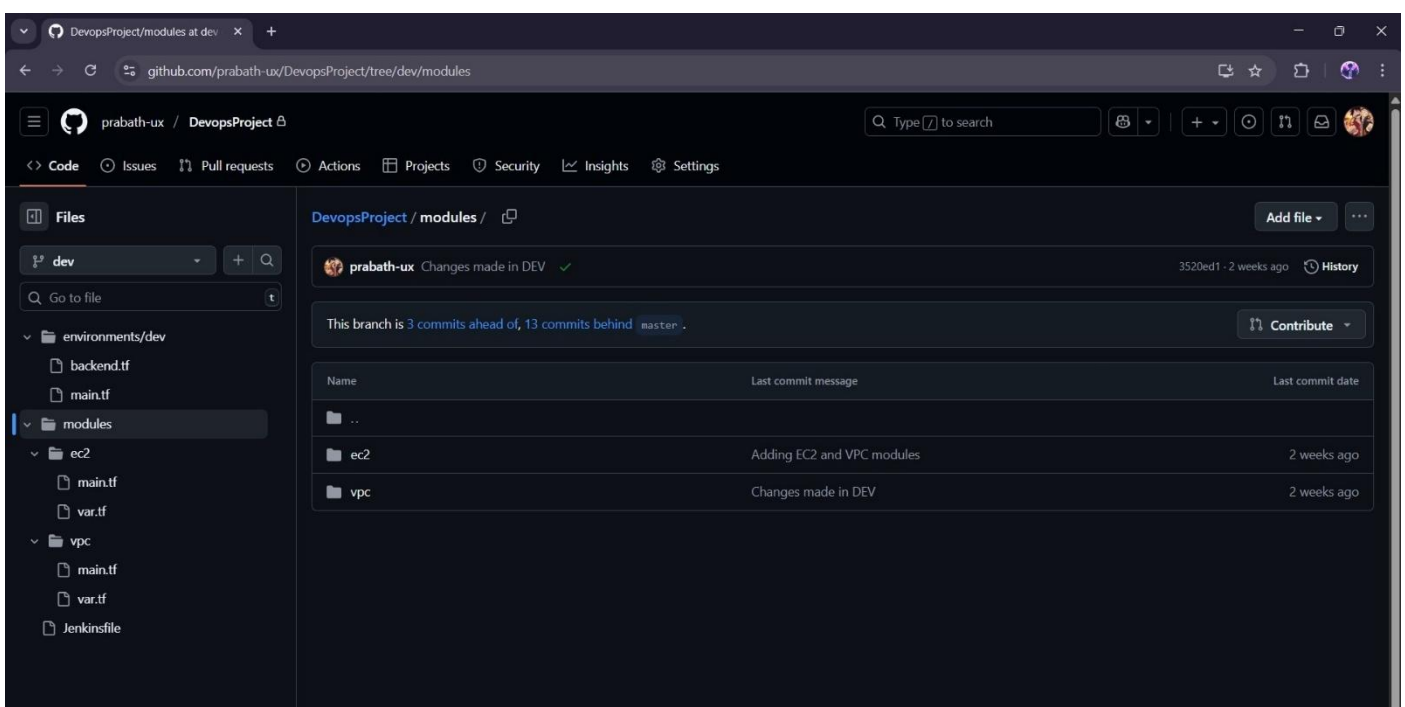
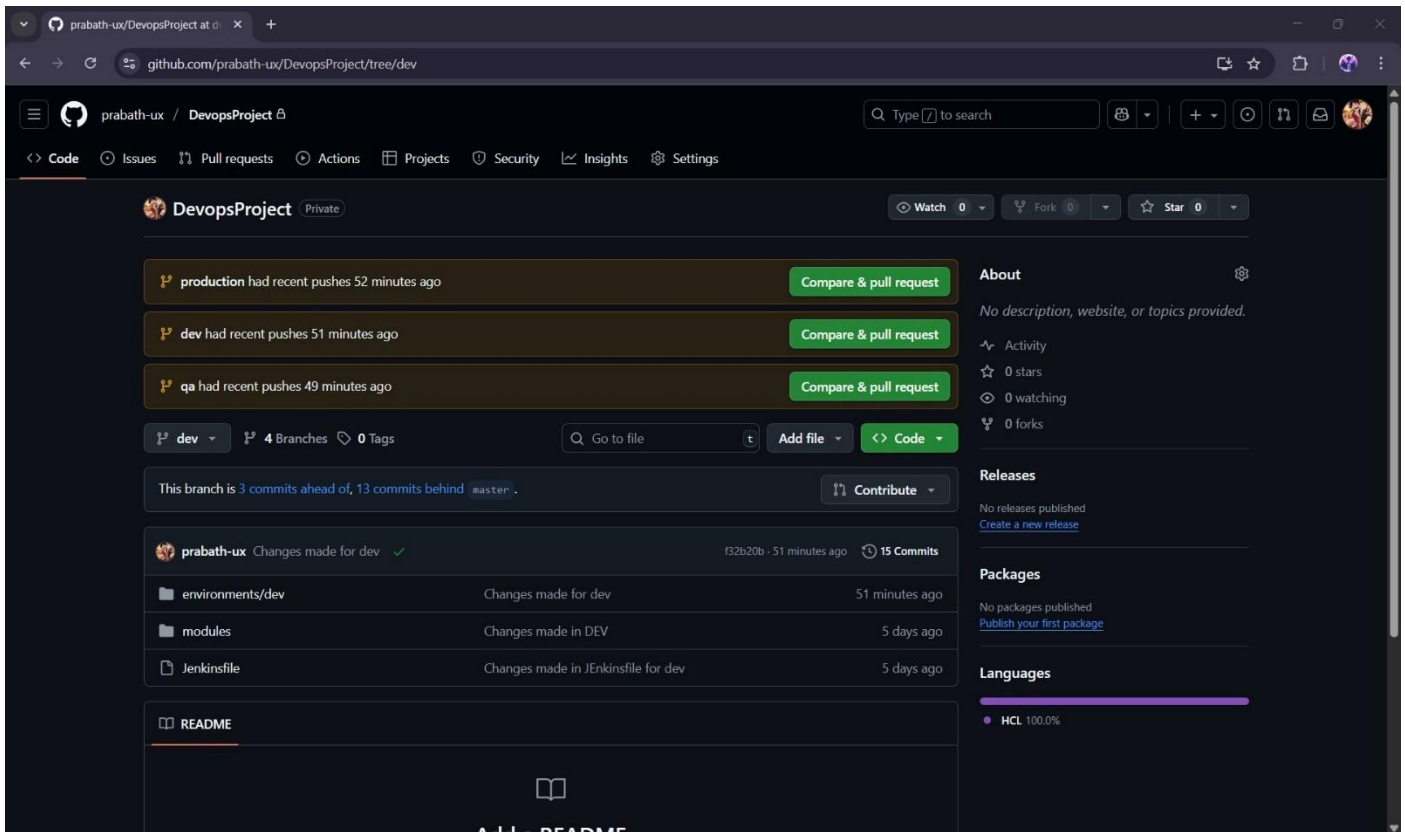
stage('Terraform Plan') {
    steps {
        dir("${TF_WORKDIR}") {
            sh 'terraform plan -out=tfplan'
            sh 'terraform show -no-color tfplan > tfplan.txt'
            sh 'cat tfplan.txt'
        }
    }
}

stage('Approval') {
    when {
        expression { env.BRANCH_NAME == 'production' }
    }
    steps {
        input message: "Approve the deployment to production?", ok: 'Deploy'
    }
}

stage('Terraform Apply') {
    steps {
        dir("${TF_WORKDIR}") {
            sh 'terraform apply tfplan'
        }
    }
}

```

- After the creation of codes give the following to command to push into the github.
- Commands:
 - git add .
 - git commit -m "The codes for dev branch"
 - git push origin dev
- And this is how it looks like in the GitHub page for dev branch.



▪ **production:**

- git checkout production

--- **backend.tf:**

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "5.100.0"  
    }  
  }  
}  
backend "s3" {  
  bucket = "devopsproject16082025"  
  key    = "environments/production/terraform.tfstate"  
  region = "ap-south-1"  
}
```

```
provider "aws" {  
  region = "ap-south-1"  
}
```

--- **main.tf:**

```
module "vpc" {  
  source      = "../modules/vpc"  
  vpc_cidr    = "12.0.0.0/16"  
  env         = "production"  
  subnet_cidr = "12.0.1.0/24"  
  az          = "ap-south-1a"  
}
```

```
module "ec2" {  
  source      = "../modules/ec2"  
  instance_type = "t3.large"  
  ami_id      = "ami-0f918f7e67a3323f0"  
  env         = "production"  
  ec2_count    = 2  
  subnet_id    = module.vpc.subnet_id  
}
```

--- **Jenkinsfile:**

```
pipeline {  
  agent any  
  
  environment {  
    ENV = "${env.BRANCH_NAME}"  
  }  
}
```

```

    TF_WORKDIR = "environments/${env.BRANCH_NAME}"
}

stages {
  stage('Checkout') {
    steps {
      git branch: "${env.BRANCH_NAME}",
        credentialsId: 'git-creds',
        url: 'https://github.com/prabath-ux/DevopsProject.git'
    }
  }

  stage('Terraform Init') {
    steps {
      dir("${TF_WORKDIR}") {
        sh 'terraform init'
      }
    }
  }

  stage('Terraform Plan') {
    steps {
      dir("${TF_WORKDIR}") {
        sh 'terraform plan -out=tfplan'
        sh 'terraform show -no-color tfplan > tfplan.txt'
        sh 'cat tfplan.txt'
      }
    }
  }

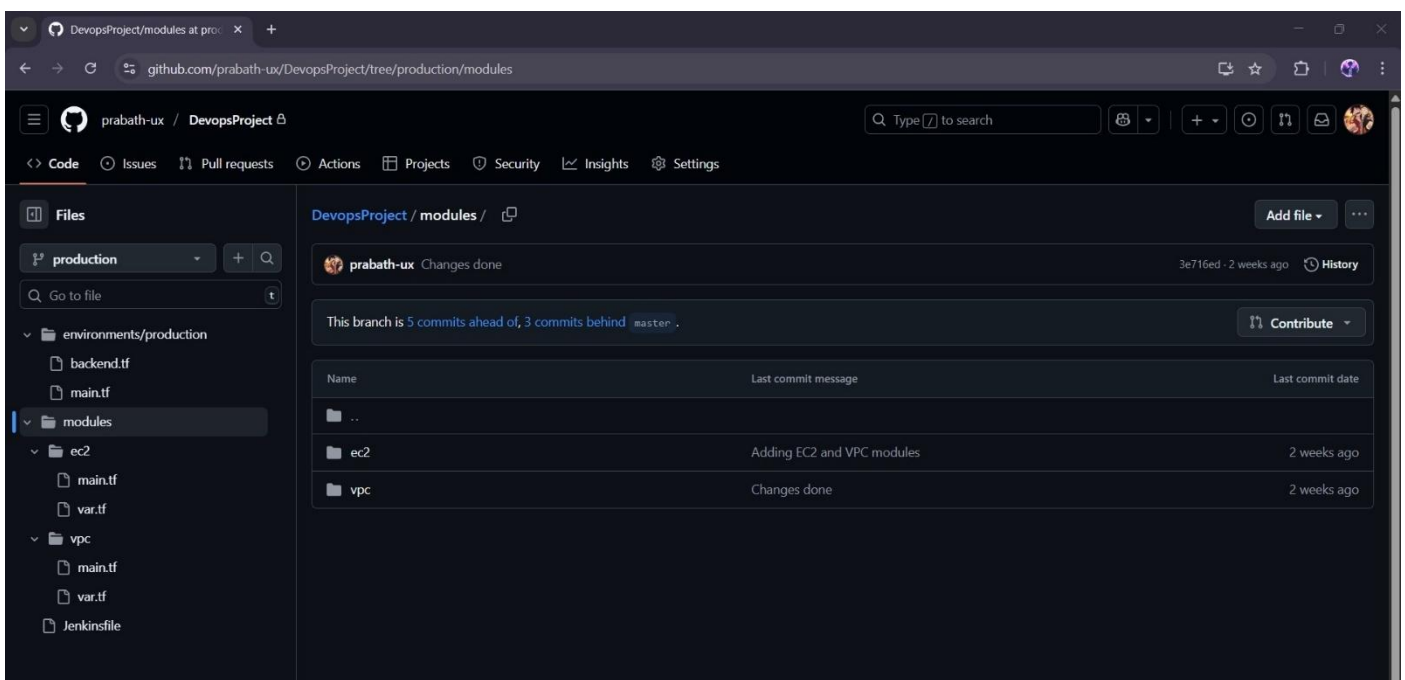
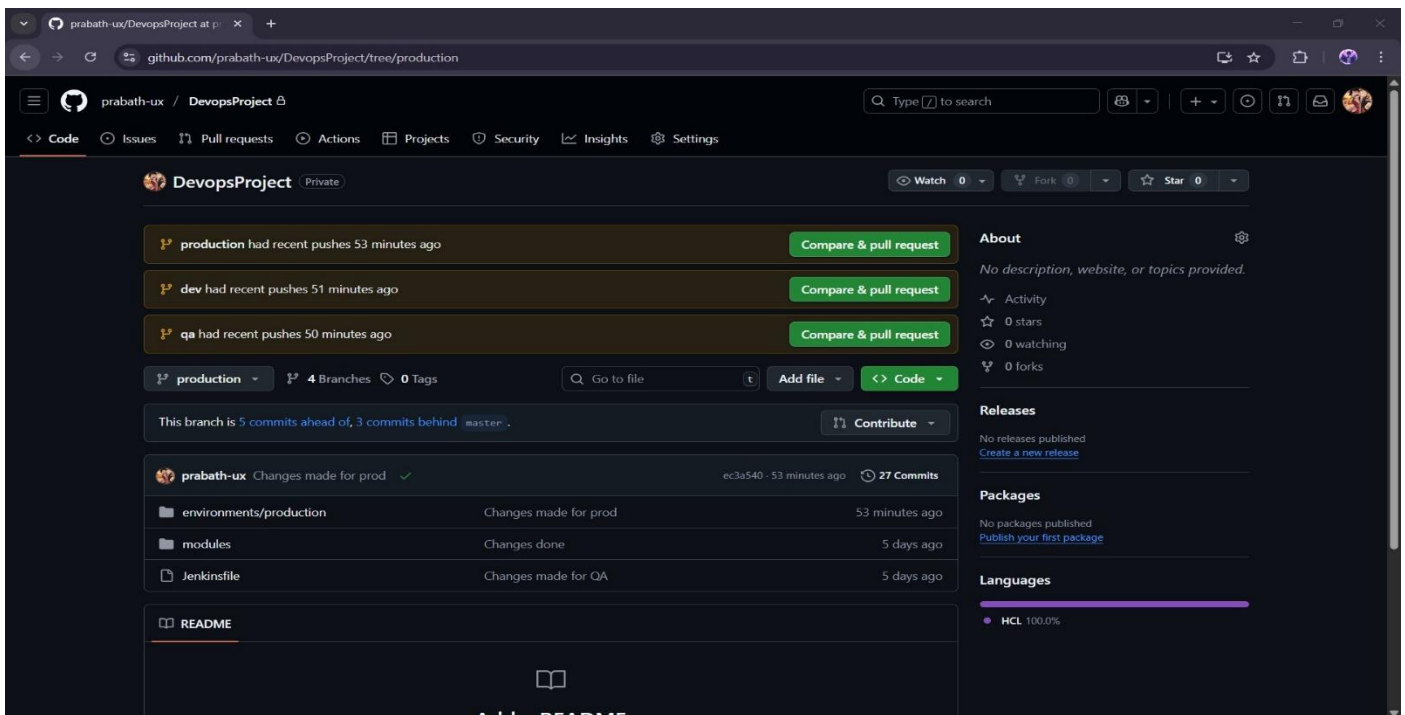
  stage('Approval') {
    when {
      expression { env.BRANCH_NAME == 'production' }
    }
    steps {
      input message: "Approve the deployment to production?", ok: 'Deploy'
    }
  }

  stage('Terraform Apply') {
    steps {
      dir("${TF_WORKDIR}") {
        sh 'terraform apply tfplan'
      }
    }
  }
}

```

```
}
}
}
}
```

- After the creation of codes give the following to command to push into the github.
- Commands:
 - git add .
 - git commit -m "The codes for production branch"
 - git push origin production
- And this is how it looks like in the GitHub page for production branch



▪ **qa:**

- git checkout qa

--- backend.tf:

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "5.100.0"
    }
  }
  backend "s3" {
    bucket = "devopsproject16082025"
    key    = "environments/qa/terraform.tfstate"
    region = "ap-south-1"
  }
}
```

```
provider "aws" {
  region = "ap-south-1"
}
```

--- main.tf:

```
module "vpc" {
  source      = "../modules/vpc"
  vpc_cidr    = "11.0.0.0/22"
  env         = "qa"
  subnet_cidr = "11.0.1.0/24"
  az          = "ap-south-1a"
}
```

```
module "ec2" {
  source      = "../modules/ec2"
  instance_type = "t2.large"
  ami_id      = "ami-0f918f7e67a3323f0"
  env         = "qa"
  ec2_count   = 2
  subnet_id   = module.vpc.subnet_id
}
```

--- Jenkinsfile:

```
pipeline {
  agent any
```

```
  environment {
```

```

ENV = "${env.BRANCH_NAME}"
TF_WORKDIR = "environments/${env.BRANCH_NAME}"
}

stages {
  stage('Checkout') {
    steps {
      git branch: "${env.BRANCH_NAME}",
        credentialsId: 'git-creds',
        url: 'https://github.com/prabath-ux/DevopsProject.git'
    }
  }

  stage('Terraform Init') {
    steps {
      dir("${TF_WORKDIR}") {
        sh 'terraform init'
      }
    }
  }

  stage('Terraform Plan') {
    steps {
      dir("${TF_WORKDIR}") {
        sh 'terraform plan -out=tfplan'
        sh 'terraform show -no-color tfplan > tfplan.txt'
        sh 'cat tfplan.txt'
      }
    }
  }

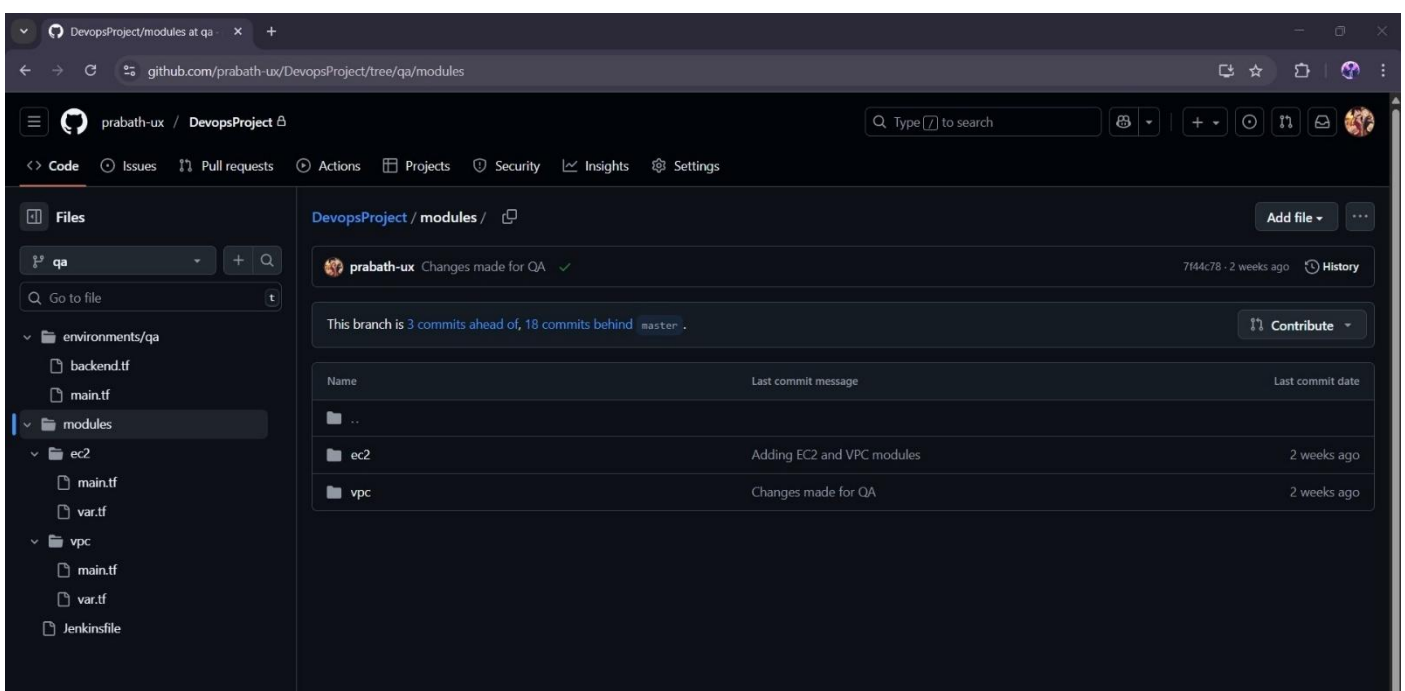
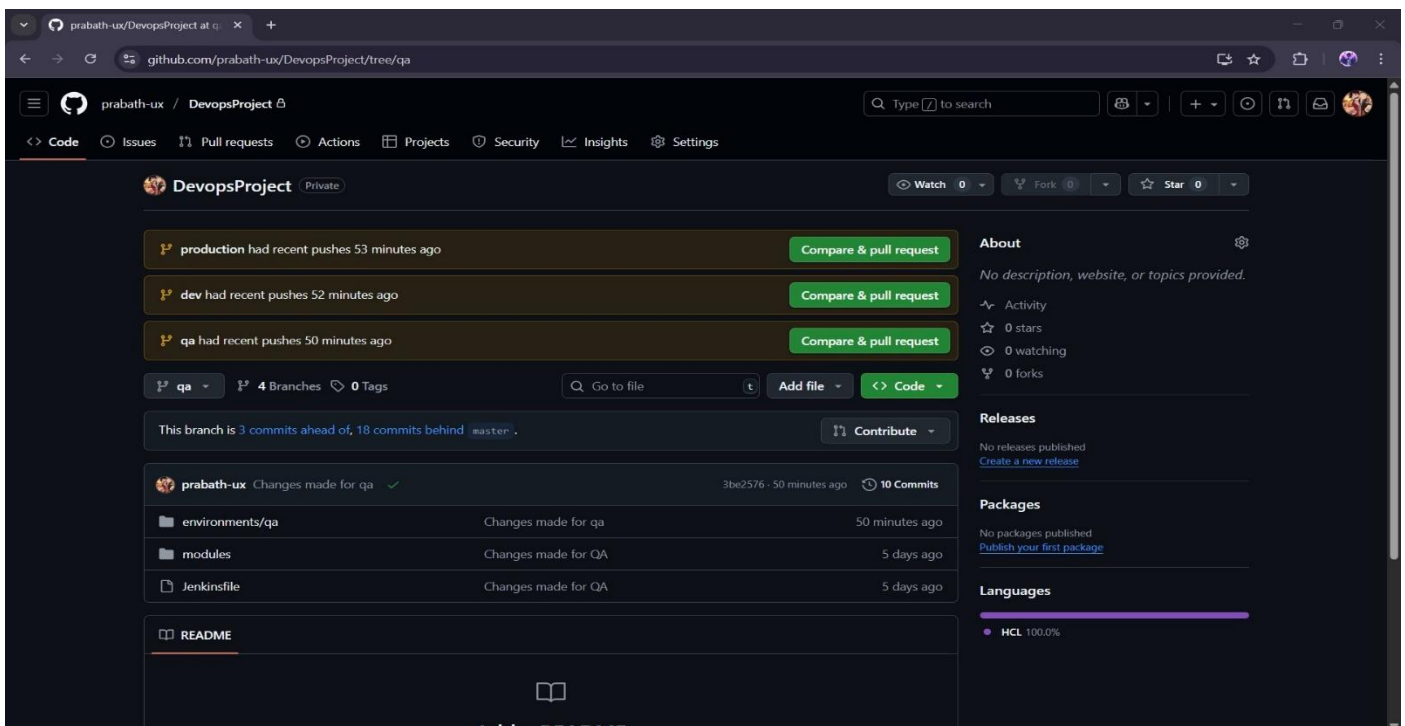
  stage('Approval') {
    when {
      expression { env.BRANCH_NAME == 'production' }
    }
    steps {
      input message: "Approve the deployment to production?", ok: 'Deploy'
    }
  }

  stage('Terraform Apply') {
    steps {
      dir("${TF_WORKDIR}") {
        sh 'terraform apply tfplan'
      }
    }
  }
}

```

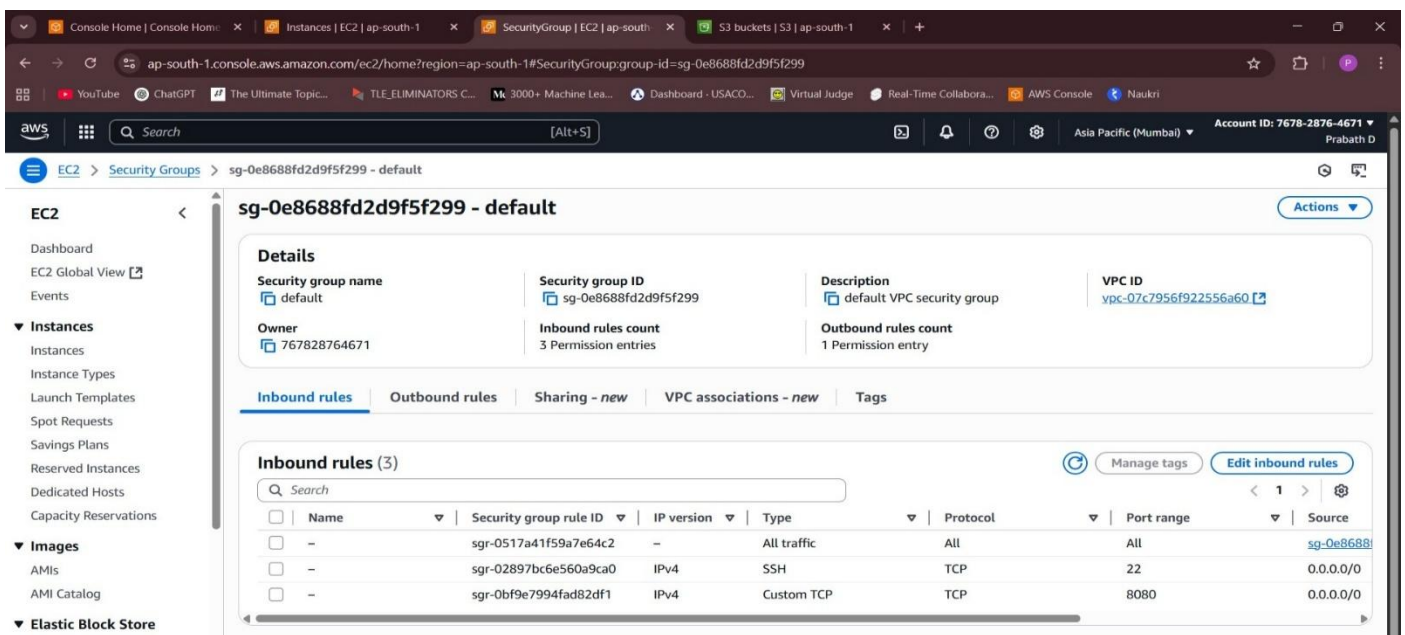
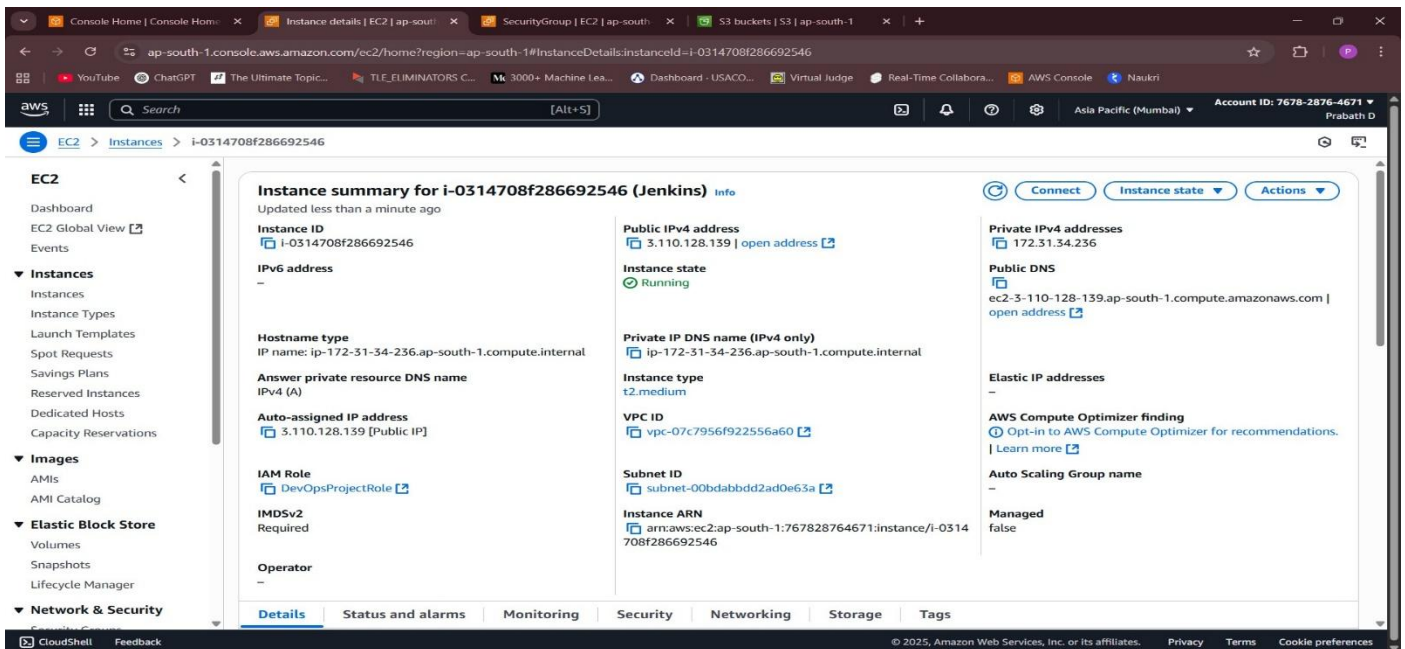
```
}
}
}
}
}
```

- After the creation of codes give the following to command to push into the github.
- Commands:
 - git add .
 - git commit -m “The codes for qa branch”
 - git push origin qa
- And this is how it looks like in the GitHub page for qa branch.



ii. Jenkins EC2, S3 & IAM Role:

- After successful creation of directories and pushed into the GitHub. Now, let's create the Jenkins setup to build and automate it.
- Go to AWS Console -> EC2 Instances -> Launch Instance -> Set the name as 'Jenkins'.
- Select the AMI Server – Ubuntu -> Select the key pair -> Select the security groups created specifically for this instance.
- Security Groups should have SSH, Custom TCP - 8080 (For Jenkins), All Traffic.
- Then create the IAM Role and attach with this instance.
- And also create a S3 Bucket name it with 'devopsproject16082025' to store the data.



DevOpsProjectRole Info Delete

Allows EC2 instances to call AWS services on your behalf.

Summary Edit

Creation date
August 10, 2025, 17:18 (UTC+05:30)

ARN
arn:aws:iam::767828764671:role/DevOpsProjectRole

Instance profile ARN
arn:aws:iam::767828764671:instance-profile/DevOpsProjectRole

Last activity
8 minutes ago

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (1) Info Simulate Remove Add permissions

You can attach up to 10 managed policies.

Filter by Type
All types

Policy name	Type	Attached entities
AdministratorAccess	AWS managed - job function	1

Permissions boundary (not set)

General purpose buckets (1) Info Copy ARN Empty Delete Create bucket

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	Creation date
devopsproject16082025	Asia Pacific (Mumbai) ap-south-1	August 16, 2025, 17:31:27 (UTC+05:30)

Account snapshot Info View dashboard

External access summary - new Info

iii. Jenkins Setup:

- After the creation of instance, copy the Public IPv4 Address of Jenkins EC2 Instance and paste it in the Putty terminal.
- Then run the following commands to install the Jenkins Server that must run in protocol - 8080.
- The following commands are:

→ Commands:

```
sudo -i
```

```
apt update
```

--For Jenkins installation, first the installation of Java needs to complete.

--Go to Jenkins.io website -> Click linux -> Scroll down to Java installation.

#java code:

```
sudo apt install fontconfig openjdk-21-jre
```

--then install the Jenkins (Latest release):

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
```

```
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
```

```
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
```

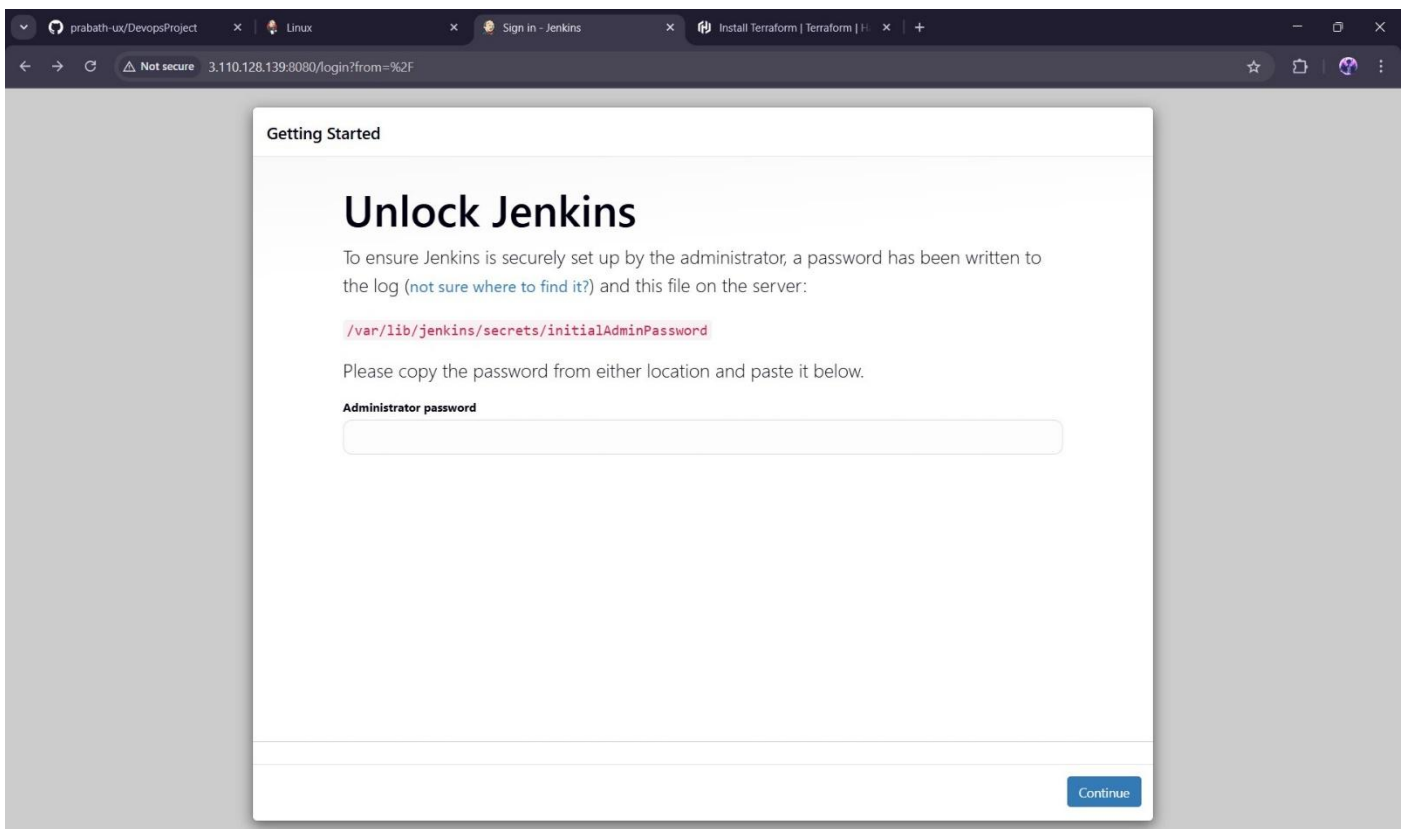
```
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
```

```
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt-get update
```

```
sudo apt-get install Jenkins
```

- Then Jenkins will be installed and now hit the Public IPv4 address by copy and pasting it in the browser the Jenkins page will appear.
- Then copy the secrets path for unlocking the password, give the command to know the password `cat <secrets-path>`. It will give the administrator password paste in this.



- Now install the plugins to run the Jenkins server in the Jenkins itself.

- Then after the installation of plugins it will redirect to admin setup. After that the Jenkins will give the Instance Configuration by having the Jenkins URL.

The screenshot shows the 'Getting Started' page of the Jenkins Setup Wizard. The title is 'Create First Admin User'. It contains five input fields: 'Username' (admin), 'Password' (masked with dots), 'Confirm password' (masked with dots), 'Full name' (Prabath), and 'E-mail address' (dvprabhu1205@gmail.com). At the bottom, there is a 'Skip and continue as admin' link and a 'Save and Continue' button. The Jenkins version 2.516.1 is displayed in the bottom left corner.

The screenshot shows the 'Getting Started' page of the Jenkins Setup Wizard. The title is 'Instance Configuration'. It features a 'Jenkins URL' input field with the value 'http://3.110.128.139:8080/'. Below the input field, there is explanatory text: 'The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps. The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.' At the bottom, there is a 'Not now' link and a 'Save and Finish' button. The Jenkins version 2.516.1 is displayed in the bottom left corner.

- Now, save and finish the dashboard will appear for the user.

- Go to settings -> Select Plugins -> Available Plugins -> Search for Pipeline Stage View -> Install it and after the installation.

The screenshot shows the Jenkins 'Available plugins' page with a search filter 'stage'. The table lists several plugins with their health status (99, 76, 76, 93, 100).

Install	Name	Released	Health
☒	Pipeline: Stage View 2.38 User Interface Pipeline Stage View Plugin.	3 mo 18 days ago	99
☐	Pipeline Aggregator View 119.v4b_ec11953552 User Interface Full screen dashboard to view stages of the pipeline as a build history. <i>This plugin is up for adoption! We are looking for new maintainers. Visit our Adopt a Plugin initiative for more information.</i>	8 mo 1 day ago	76
☐	Job and Stage monitoring 3.6.2 Watches pipeline jobs and provides job and stage stats such as time and pass/fail. Can be configured to update GitHub commit status (one status per stage) and send stats to an InfluxDB instance, or StatsD collector, for build health monitoring of job/stage timing and success rate.	5 yr 5 mo ago	76
☐	Splunk Plugin Extension 1.11.0 This plugin extracts pipeline job stages information	2 mo 24 days ago	93
☐	Pipeline As YAML (Incubated) 270.vc348a_45d9214 pipeline Enables defining Jenkins Pipelines as YAML Files. This plugin is currently in the Incubation Stage (see the documentation).	5 mo 15 days ago	100
☐	Web Security Application Project (WSAP) 1.0 WSAP for Jenkins pretends to identifying security vulnerabilities of Web Applications in CI/CD development The Web Security Application Project (WSAP) solution pretends to enhance the CI/CD		

The screenshot shows the Jenkins 'Download progress' page. A list of plugins is shown with a 'Success' status for each. At the bottom, there are instructions to go back to the top page and a checkbox to restart Jenkins.

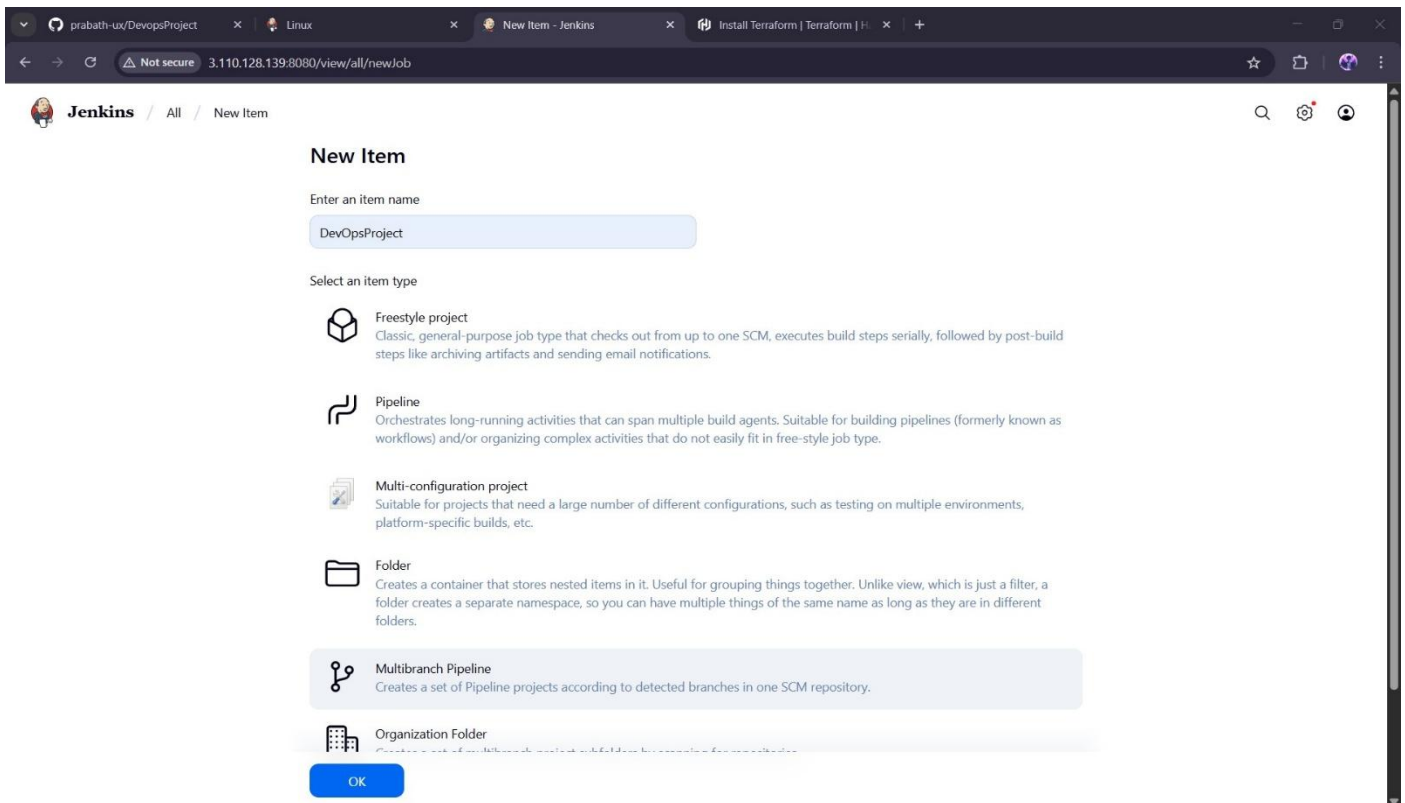
Pipeline: GitHub Groovy Libraries	Success
Metrics	Success
Pipeline Graph View	Success
Git	Success
EDDSA API	Success
Trilead API	Success
SSH Build Agents	Success
Matrix Authorization Strategy	Success
LDAP	Success
jsoup API	Success
Email Extension	Success
Mailer	Success
Theme Manager	Success
Dark Theme	Success
Loading plugin extensions	Success
Pipeline Graph Analysis	Success
Pipeline: REST API	Success
Pipeline: Stage View	Success
Loading plugin extensions	Success

→ [Go back to the top page](#)
(you can start using the installed plugins right away)

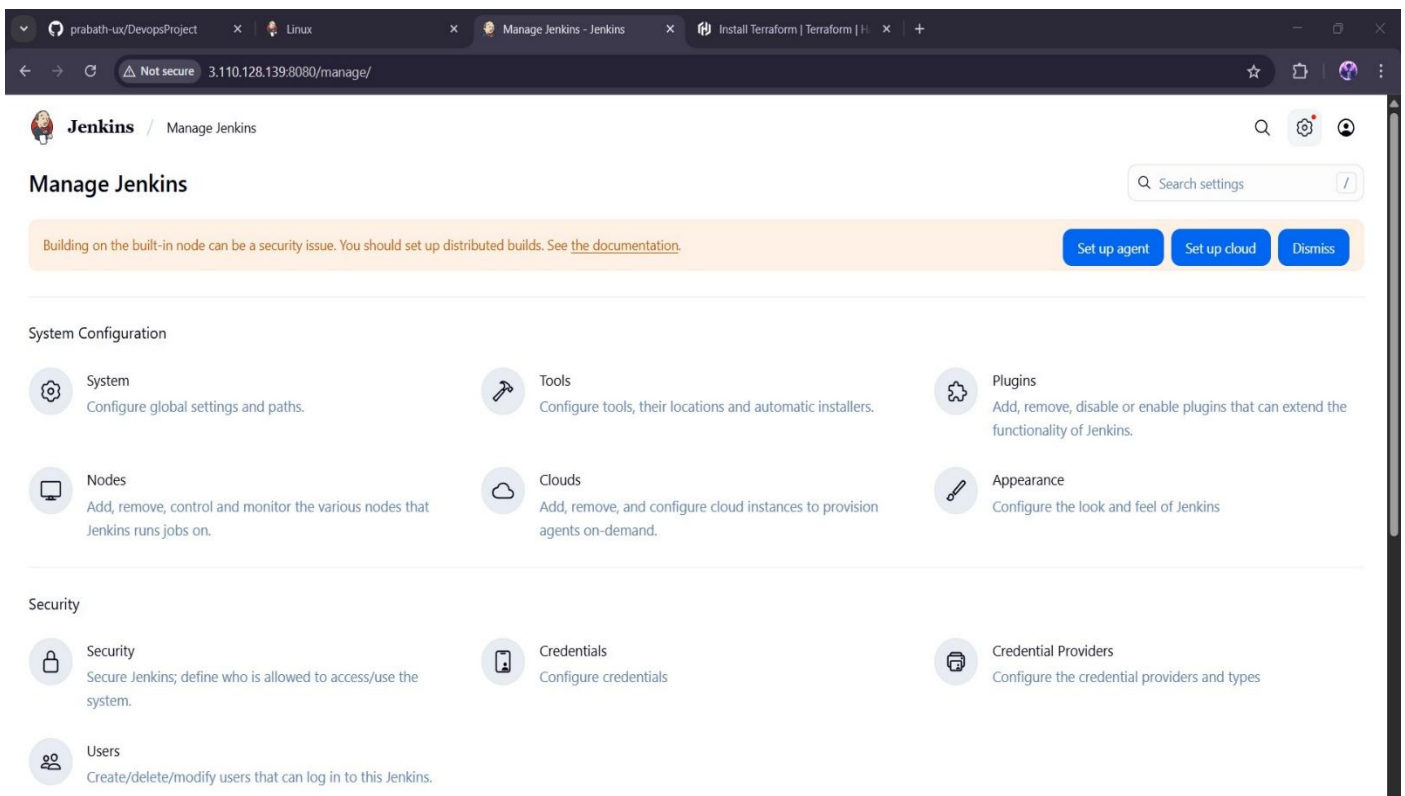
→ ☐ Restart Jenkins when installation is complete and no jobs are running

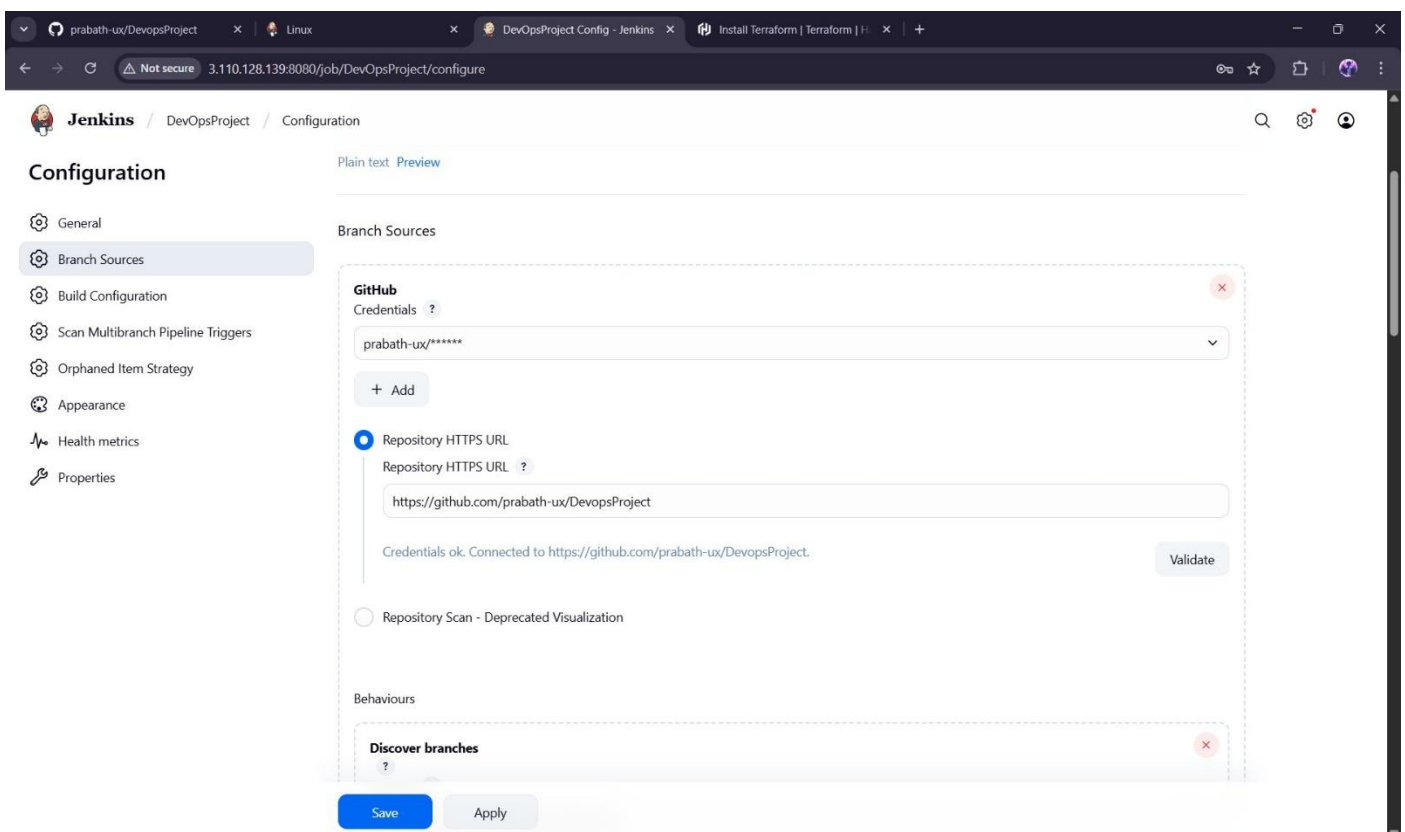
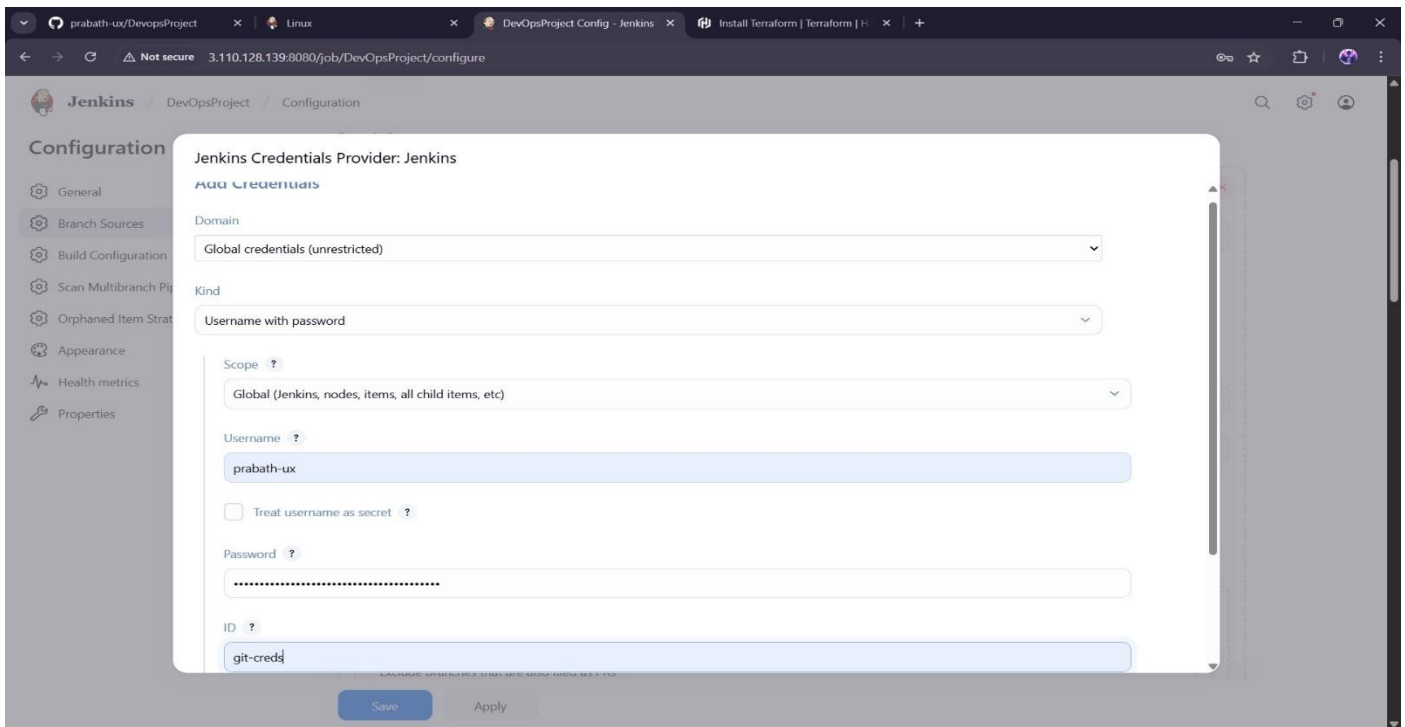
REST API Jenkins 2.516.1

- Go to New Item -> Give the name as DevopsProject -> Select Multi-Branch Pipeline -> Give Ok.



- To trigger the actions in the Jenkins first the user need to give credentials.
- So, for that the user has to initiate the webhook for this. Go to GitHub -> Settings -> Developer Settings -> Webhooks -> Give name -> payload URL – Jenkins URL/github.com/webhook/ -> This will initiate the triggering process and it will show the green tick and shows ‘successfully triggered message’.
- Now, go to Jenkins -> Settings -> Credentials -> Add credentials -> Select Username and Password -> Click add branch -> Select for Jenkins -> give access name -> PAT for password -> id – git-creds -> click save.





iv. Build Configurations in Jenkins:

- After successful completion of everything the branch will run automatically for all the branch – dev, qa and production
- The only thing which we need to approve is that during the production branch in the approval stage, that we initiated only for production branch Jenkinsfile code.

- So, the following outputs of dev, qa and production will give the clarity about it.

prabath-ux/DevOpsProject x Linux x dev [DevOpsProject] - Jenkins x Install Terraform | Terraform | H x +

← → ↻ 3.110.128.139:8080/job/DevOpsProject/job/dev/ ☆ 📄 👤

Jenkins / DevOpsProject / dev

Status

Changes

Build Now

View Configuration

Full Stage View

Stages

GitHub

Pipeline Syntax

Builds

Filter /

Today

#1 12:43 pm

dev

Full project name: DevOpsProject/dev

Stage View

Average stage times:
(full run time: ~1min 41s)

	Declarative: Checkout SCM	Checkout	Terraform Init	Terraform Plan	Approval	Terraform Apply
#1	5s	2s	22s	15s	0ms	48s

Aug 16 18:13 No Changes

Permalinks

- Last build (#1), 19 sec ago

prabath-ux/DevOpsProject x Linux x qa [DevOpsProject] - Jenkins x Install Terraform | Terraform | H x +

← → ↻ 3.110.128.139:8080/job/DevOpsProject/job/qa/ ☆ 📄 👤

Jenkins / DevOpsProject / qa

Status

Changes

Build Now

View Configuration

Full Stage View

Stages

GitHub

Pipeline Syntax

Builds

Filter /

Today

#1 12:43 pm

qa

Full project name: DevOpsProject/qa

Stage View

Average stage times:
(full run time: ~2min 40s)

	Declarative: Checkout SCM	Checkout	Terraform Init	Terraform Plan	Approval	Terraform Apply
#1	2s	2s	10s	13s	0ms	29s

Aug 16 18:13 No Changes

Permalinks

- Last build (#1), 2 min 59 sec ago
- Last stable build (#1), 2 min 59 sec ago
- Last successful build (#1), 2 min 59 sec ago
- Last completed build (#1), 2 min 59 sec ago

prabath-ux/DevOpsProject x Linux x production [DevOpsProject] - Jenkins x Install Terraform | Terraform | H x +

← → ↻ 3.110.128.139:8080/job/DevOpsProject/job/production/ ☆ 📄 👤

Jenkins / DevOpsProject / production

Status

Changes

Build Now

View Configuration

Full Stage View

Stages

GitHub

Pipeline Syntax

Builds

Filter /

Today

#1 12:43 pm

production

Full project name: DevOpsProject/production

Stage View

Average stage times:
(full run time: ~2min 39s)

	Declarative: Checkout SCM	Checkout	Terraform Init	Terraform Plan	Approval	Terraform Apply
#1	2s	2s	22s	18s	139ms	19s

Aug 16 18:13 No Changes

Permalinks

- Last build (#1), 2 min 14 sec ago

The screenshot shows the Jenkins web interface for a project named 'DevOpsProject'. The left sidebar contains navigation options like Status, Configure, Scan Repository Now, Scan Repository Log, Multibranch Pipeline Events, Delete Multibranch Pipeline, Build History, Project Relationship, Check File Fingerprint, GitHub, Rename, Pipeline Syntax, and Credentials. The main area displays the 'Branches (3)' tab, showing a table of build statuses for 'dev', 'production', and 'qa' branches. All branches show a 'Last Success' status with a green checkmark and a duration of 12 minutes. The 'dev' branch has a duration of 1 min 41 sec, 'production' has 2 min 39 sec, and 'qa' has 2 min 40 sec. Below the table, there are sections for 'Build Queue' (No builds in the queue) and 'Build Executor Status' (0/2). The bottom right corner indicates 'REST API' and 'Jenkins 2.516.1'.

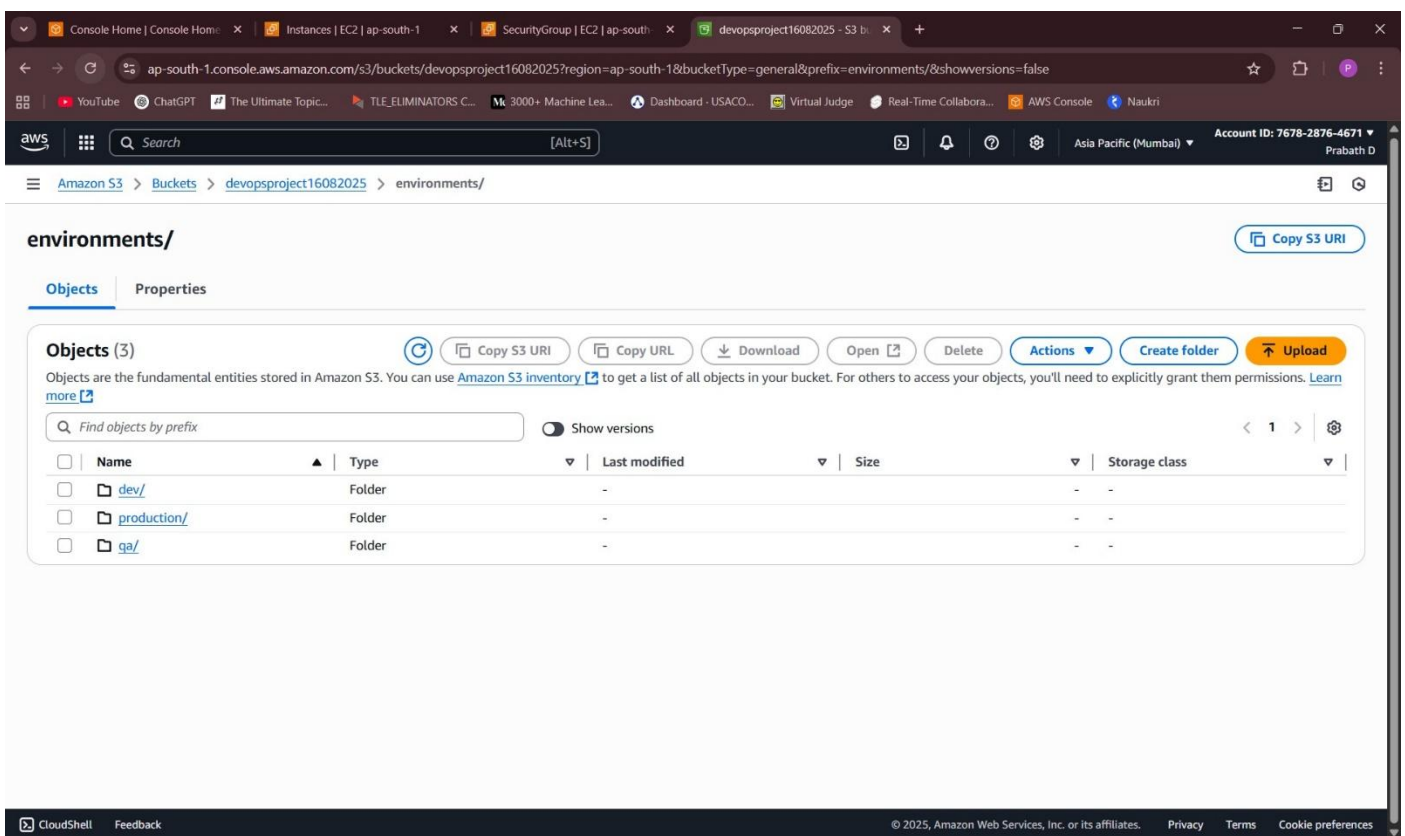
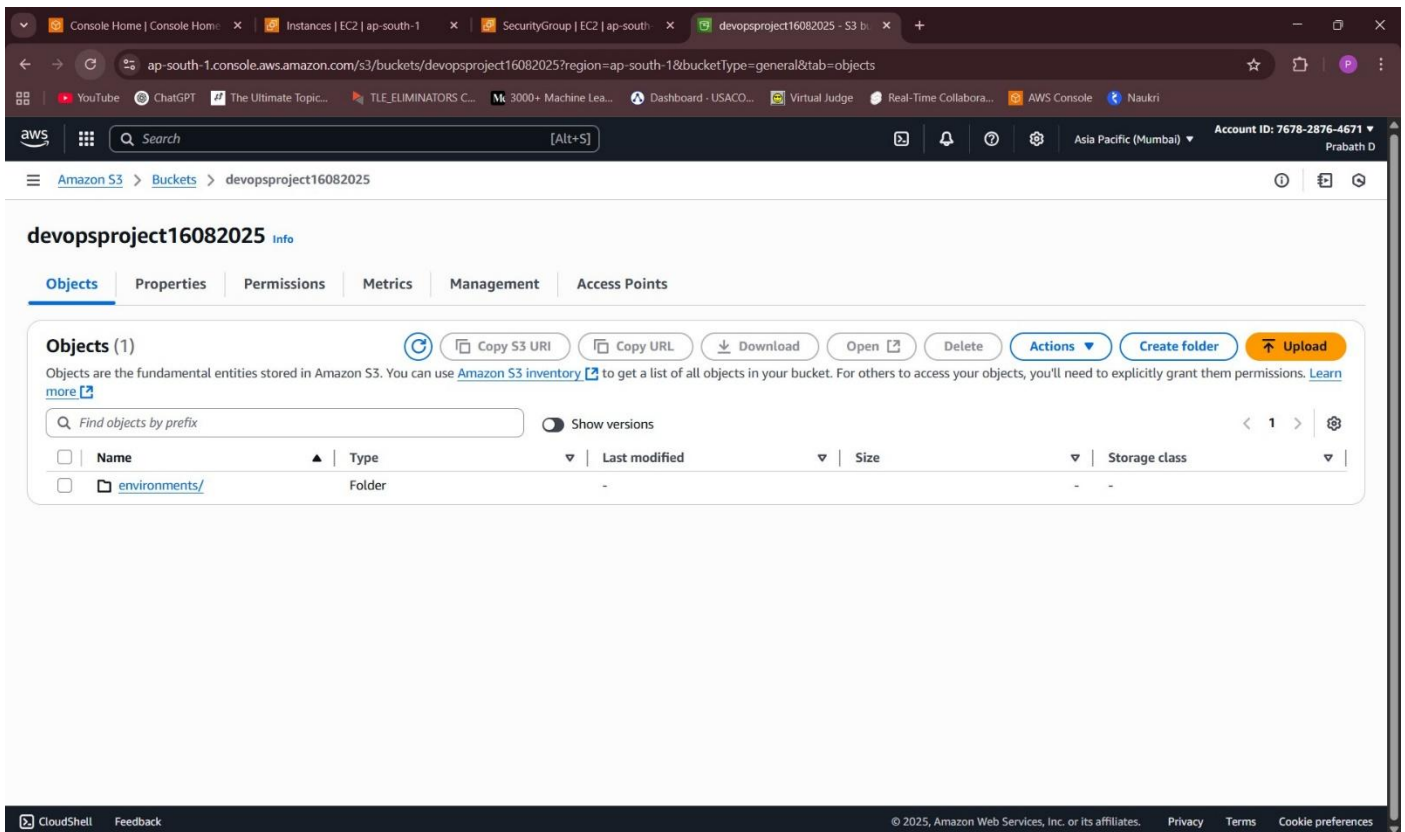
S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	dev	12 min #1	N/A	1 min 41 sec
✓	☀	production	12 min #1	N/A	2 min 39 sec
✓	☀	qa	12 min #1	N/A	2 min 40 sec

- Now, the all the branches have been built.
- To check if the instances has created or not. The user need to go to EC2 Instances in AWS Console.
- Then the user can verify by knowing about the instances created.

The screenshot shows the AWS Management Console 'Instances' page. The left sidebar shows the navigation menu with 'EC2' selected. The main area displays a list of 8 EC2 instances. The instances are: dev-EC2-0, dev-EC2-1, dev-EC2-2, Jenkins, production-EC2-0, production-EC2-1, and qa-EC2-0. All instances are in a 'Running' state. The 'Jenkins' instance is highlighted with a blue border. The table columns include Name, Instance ID, Instance state, Instance type, Status check, Alarm status, Availability Zone, and Public IPv4.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
dev-EC2-0	i-0bc8ad4cc5981882c	Running	t2.medium	2/2 checks passed	View alarms +	ap-south-1a	-
dev-EC2-1	i-09dccc6d65ab032838	Running	t2.medium	2/2 checks passed	View alarms +	ap-south-1a	-
dev-EC2-2	i-051805c444cea2eab	Running	t2.medium	2/2 checks passed	View alarms +	ap-south-1a	-
Jenkins	i-0314708f286692546	Running	t2.medium	2/2 checks passed	View alarms +	ap-south-1a	ec2-3-110-1
production-EC2-0	i-0d261d79afc53b907	Running	t3.large	3/3 checks passed	View alarms +	ap-south-1a	-
production-EC2-1	i-0fb604020852d21d6	Running	t3.large	3/3 checks passed	View alarms +	ap-south-1a	-
qa-EC2-0	i-06c8250e7aaf0d9ad	Running	t2.large	2/2 checks passed	View alarms +	ap-south-1a	-

- Also, the user can check in the S3 Bucket for the created files that is getting stored.



✦ With that the Infra-Deployment Pipeline is implemented by creating the Multi-Branch Pipeline for Dev, QA, and Production.